

Fuerza Bruta

Fuerza Bruta

Fuerza Bruta

La búsqueda por fuerza bruta, búsqueda combinatoria, búsqueda exhaustiva o simplemente fuerza bruta, es una técnica trivial pero a menudo usada, que consiste en enumerar sistemáticamente todos los posibles candidatos para la solución de un problema, con el fin de chequear si dicho candidato satisface la solución al mismo.

Algoritmos Greedy

Algoritmos Greedy

Técnica sencilla para resolver problemas

Normalmente son algoritmos que resuelven problemas de **optimización**:

- **Camino mas corto** que une dos ciudades.

Algoritmos Greedy

Técnica sencilla para resolver problemas

Normalmente son algoritmos que resuelven problemas de **optimización**:

- **Camino mas corto** que une dos ciudades.
- **Valor máximo** alcanzable entre ciertos objetos.

Algoritmos Greedy

Técnica sencilla para resolver problemas

Normalmente son algoritmos que resuelven problemas de **optimización**:

- **Camino mas corto** que une dos ciudades.
- **Valor máximo** alcanzable entre ciertos objetos.
- **Costo mínimo** para proveer cierto servicio.

Algoritmos Greedy

Técnica sencilla para resolver problemas

Normalmente son algoritmos que resuelven problemas de **optimización**:

- **Camino mas corto** que une dos ciudades.
- **Valor máximo** alcanzable entre ciertos objetos.
- **Costo mínimo** para proveer cierto servicio.
- **Menor numero de billetes** para pagar importe.

Algoritmos Greedy

Técnica sencilla para resolver problemas

Normalmente son algoritmos que resuelven problemas de **optimización**:

- **Camino mas corto** que une dos ciudades.
- **Valor máximo** alcanzable entre ciertos objetos.
- **Costo mínimo** para proveer cierto servicio.
- **Menor numero de billetes** para pagar importe.
- **Menor tiempo** necesario para realizar un trabajo

Los algoritmos voraces intentan construir la solución optima buscando **paso a paso**

Elige la **mejor opción que parece apropiada** en cada paso

Con la esperanza de llegar a una solución general optima.

Algoritmos Greedy > Características

- Nunca revisa una **selección** ya realizada.
- Confían en seleccionar la mejor opción.
- No todos los problemas tienen solución Greedy.
- Pero muchos problemas si permiten estas soluciones.

Algoritmos Greedy > Características

- Se dispone de un conjunto **C Candidatos** de los cuales se selecciona un subconjunto que optimice alguna propiedad.
- A medida que avanza el algoritmo se selecciona candidatos y se coloca en otro subconjunto **S Seleccionados** o **R Rechazados**
- Existe una función que determina si un elemento es un candidato **optimo** para la solución (No necesariamente optima del problema)
- Existe una función que devuelve el candidato mas **optimo**

Algoritmos Greedy > Idea General

```
1. C ::= conjunto de candidatos; S ::= {}  
2. WHILE (C != {} and ! esSolucion(S))  
3.   x ::= seleccion(C); C ::= C - {x}  
4.   IF esViable(S + {x})  
5.     S ::= S + {x}  
6.   ENDIF  
7. ENDWHILE  
8. IF esSolucion(S)  
9.   RETURN S  
10. ELSE RETURN  
11.   "No encontré soluciones"  
12. ENDIF  
13.
```

Ejemplo

Se tienen **n objetos** (cada objeto **i** tiene un peso w_i y un valor v_i) y una mochila con capacidad máxima de **w**. Se pretende encontrar la manera de cargar la mochila de forma que se **maximice** el valor de lo transportado y se respete su capacidad **máxima**.

$\text{MAX} \left(\sum_{i=1}^n x_i v_i \right)$ Si y solo si No exceda la capacidad **máxima**

Ejemplo

Se tienen **n objetos** (cada objeto i tiene un peso w_i y un valor v_i) y una mochila con capacidad máxima de w . Se pretende encontrar la manera de cargar la mochila de forma que se **maximice** el valor de lo transportado y se respete su capacidad **máxima**.

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

```
int[] w = {10, 20, 30, 40, 50};
```

```
int[] v = {20, 30, 66, 40, 60};
```

T

```
1. public static double[] greedy(int[] weights, int[] values, int capacity) {
2.     double[] solutions = new double[weights.length];
3.     double currentWeight = 0;
4.
5.     while (currentWeight < capacity) {
6.         int i = select(weights, values, solutions ); //Escoge la mejor solución
7.         if (i == -1) break;
8.
9.         if (currentWeight + weights[i] <= capacity) {
10.             solutions[i] = 1;
11.             currentWeight += weights[i];
12.         } else {
13.             solutions[i] = (capacity - currentWeight) / (double) weights[i];
14.             break;
15.         }
16.     }
17.
18.     return solutions;
19. }
20.
```

Que hace?

Datos de entrada:

- Arreglos **w[1..n]**, y **v[1..n]** contienen con los **pesos** y **valores** de los **objetos**.

Datos de salida:

- Arreglo **solution[1..n]** con la porción de cada elemento que se carga en la mochila.
 - **solution**_[i] = 1 significa que el objeto **i** se lleva completo
 - **solution**_[i] = 0 que nada se lleva del objeto **i**
 - **solution**_[i] = r ; $0 < r < 1$ significa que el elemento **i** se lleva fraccionado.

Que ruta seguir?

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

1. Seleccionar el elemento ...

Algoritmos Greedy– Mayor Valor

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

1. Seleccionar el elemento de **mayor valor**

Algoritmos Greedy– Mayor Valor

1. Seleccionar el elemento de **mayor valor**

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

El peso (**w**) es menor que la capacidad de la mochila? **si**

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

Capacidad de la mochila = $100 - 30 = 70$

El peso (**w**) (50) es menor que la capacidad de la mochila? **si**

Capacidad de la mochila = $70 - 50 = 20$

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

Algoritmos Greedy– Mayor Valor

Capacidad de la mochila = $70 - 50 = 20$

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	60 1	40 3	60 2

La mochila solo puede soportar 20, pero siguiendo la lógica el máximo es **40**

Entonces tomamos 20 - 40(el siguiente máximo valor)

Entonces en la mochila podemos llevar un **máximo de 146** valores sin pasar la capacidad máxima que soporta la mochila **w = 100**

Algoritmos Greedy– Menor Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

2. Seleccionar el elemento de **menor peso**

(Similar al anterior, ahora seleccionamos el que tiene menor peso)

Algoritmos Greedy– Menor Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

2. Seleccionar el elemento de **menor peso**

(Similar al anterior, ahora seleccionamos el que tiene menor peso)

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10 	20 	30 	40 	50
v	20	30	66	40	60

Entonces podemos llevar un **máximo de 156**

Algoritmos Greedy–Mayor Valor por unidad de Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

1. Seleccionar el elemento de **mayor valor**
2. Seleccionar el elemento de **menor peso**
3. Seleccionar el elemento que tenga **mayor valor por unidad de peso**

Algoritmos Greedy–Mayor Valor por unidad de Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

	1	2	3	4	5
	20/10	30/20	66/30	40/40	60/50
	2	1.5	2.2	1	1.2

Algoritmos Greedy–Mayor Valor por unidad de Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

	Obj. 1	Obj. 2	Obj. 3	Obj. 4	Obj. 5
	2	1.5	2.2	1	1.2

Algoritmos Greedy–Mayor Valor por unidad de Peso

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5
w	10	20	30	40	50
v	20	30	66	40	60

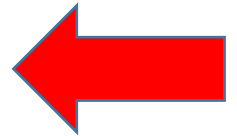
	Obj. 1	Obj. 2	Obj. 3	Obj. 4	Obj. 5
	2	1.5	2.2	1	1.2

$$W = 30 + 10 + 20 + \frac{40(\text{peso faltante})}{50(\text{peso objeto})} = 100$$

$$V = 66 + 20 + 30 + (60 \cdot 0.8) = 164$$

Algoritmos Greedy– Resumen del ejemplo

	obj. 1	obj. 2	obj. 3	obj. 4	obj. 5	
Max v_i	0	0	1	0,5	1	146
Min w_i	1	1	1	1	0	156
Max v_i/w_i	1	1	1	0	0,8	164



Códigos

1. Seleccionar el elemento de **mayor valor**

`greedy_mochila_menor_valor.java`

2. Seleccionar el elemento de **menor peso**

`greedy_mochila_menor_peso.java`

3. Seleccionar el elemento que tenga **mayor valor por unidad de peso**

`greedy_mochila_maximo_valor_peso.java`

Ejemplo

Se deben generar 36 monedas y se requiere gastar el menor numero de monetas (hay monedas de **1, 2, 5, 10 y 20**)

Ejemplo

Criterio sera elegir la moneda mas alta posible?

Se deben generar 36 pesos y se requiere **gastar el menor numero de monetas** (hay monedas de 1, 2, 5, 10 y 20)

Selecciono la mas alta moneda **20 -> Faltal 16**

Se deben generar **16** pesos y se requiere **gastar el menor numero de monetas** (hay monedas de 1, 2, 5, 10)

Selecciono la mas alta moneda **10 -> Faltal 6**

Se deben generar **6** monedas y se requiere **gastar el menor numero de monetas** (hay monedas de 1, 2, 5)

Selecciono la mas alta moneda **5-> Faltal 1**

Se deben generar **1** monedas y se requiere **gastar el menor numero de monetas** (hay monedas de 1, 2)

Se puede usar la moneda de **1** -> **Falta 0**

Entonces use 4 monedas (20, 10, 5 y 1)

Es una buena solucion

Se deben generar **36** pesos y se requiere **gastar el menor numero de monetas** (hay monedas de 1, 5, 10, 18, 20)

Teniendo 36 pesos, se puede usar monedas de 20

Falta 16 pesos

Teniendo 16 pesos, se puede usar una moneda de 10

Falta 6 pesos

Teniendo 6 pesos, se puede usar una moneda de 5

Falta 1 peso

Teniendo 1 peso, se puede usar una moneda de 1;

Falta 0

Se gasto 4 monedas (20, 10, 5, 1)

Pero si uso dos moendas de 18 entonces mi solucion optima es 2 monedas

Pero que pasa si probamos con este caso
Debo generar 36 monedas (1, 2, 5, 10, 18 y 20)
Siguiendo la misma logica



Pero que pasa si probamos con este caso
Debo generar 36 monedas (1, 2, 5, 10, 18 y 20)
Siguiendo la misma logica

36
16
6
1
0

0
20
10
5
1

Pero que pasa si probamos con este caso
Debo generar 36 monedas (1, 2, 5, 10, 18 y 20)
Siguiendo la misma lógica

Sera el mas optimo?

Pero que pasa si probamos con este caso
Debo generar 36 monedas (1, 2, 5, 10, 18 y 20)
Siguiendo la misma lógica

Que pasa si usamos dos monedas de **18**?

Pero que pasa si probamos con este caso
Debo generar 36 monedas (1, 2, 5, 10, 18 y 20)
Siguiendo la misma lógica

Que pasa si usamos dos monedas de **18**?

Resulta ser el mas optimo

Problemas

<https://vjudge.net/contest/687713>

